Nigel Tendo

Legand Burge

Software Engineering

6 March 2026

Code Review Summary Report

Overview

The assignment required building a Boggle solver in Python using object-oriented design. The program accepts an NxN letter grid and a word dictionary, then identifies all valid words formed by connecting adjacent tiles without reuse. My implementation uses a Boggle class with a depth-first search algorithm and prefix-pruning optimization that terminates unproductive paths early using a precomputed prefix set, keeping all search logic private behind a clean public interface.

Feedback Received

My review partner left twenty inline comments addressing naming, validation, and clarity. The primary concerns were camelCase method names "(setGrid, getSolution)" violating PEP 8 snake_case convention, a bare integer literal 3 used as the minimum word length instead of a named constant, missing rectangular-grid validation, the absence of .strip() in dictionary normalization, a never-read _solution attribute, and a multi-condition boundary check in "_dfs" that was difficult to scan at a glance.

Feedback Given

Reviewing my partner's code, I identified five issues. The most critical was missing backtracking: the visited matrix was not reset after each DFS branch, causing the solver to miss large numbers of valid words. I also flagged inconsistent case normalization producing false negatives, the absence of prefix pruning making traversal unnecessarily slow on large grids, quadratic string concatenation inside the recursive call accumulating $O(n^2)$ cost per path, and helper state stored in module-level variables rather than encapsulated in the class.

Improvements Implemented

I applied seven changes in response to feedback. All public methods were renamed snake_case. The minimum word length was extracted to MIN_WORD_LEN = 3. A "_validate_grid()" helper was added to raise a descriptive "ValueError" for empty or non-rectangular grids. Dictionary normalization "gained .strip()" and an ".isalpha()" filter. The

unused _solution attribute was removed. The boundary check was extracted into an "_in_bounds()" helper. Neighbor offsets were precomputed as a module-level NEIGHBOR_DELTAS constant, and the duplicate prefix-building logic was consolidated into a "_build_prefixes()" method.

Static Analysis Results

Running pylint on the original file produced a score of 7.1/10 with fourteen warnings. C0103 flagged camelCase method names; C0326 reported mixed two- and four-space indentation in setter bodies; W0612 identified _solution as assigned but never used; and W1514 flagged the bare integer 3 as a magic value. All four were resolved in the revised file. One remaining warning, R0201 noting that "_expand_tile" could be a static method, was intentionally retained to preserve subclass extensibility. The revised file scored 9.4/10.

Regression Testing

Before refactoring, I captured the baseline output of "get_solution" on the provided 4x4 grid: ART, ARTY, EGO, GENT, GET, NET, NEW, NEWT, NOT, PRAT, PRY, QUA, QUART, QUARTZ, RAT, TAR, TARP, TEN, WENT, WET. After each change, I compared the output against this list and observed no differences. I then ran four edge-case checks: an empty grid and a jagged grid each now raise a "ValueError" with a descriptive message, a whitespace-padded dictionary entry matches correctly after strip normalization, and a non-alphabetic token is silently filtered out.

Reflection

This exercise highlighted that naming consistency, clear comments, and validated inputs are not aesthetic choices but professional obligations. The camelCase methods in my original code were caught immediately by a second reader even though I had stopped noticing them, illustrating why peer review is irreplaceable. Giving feedback was equally instructive: tracing the missing backtracking in my partner's code required mentally simulating the algorithm and deepened my own understanding of DFS correctness. The process reinforced that writing code for a reader, not just a compiler, is a discipline that improves practice.